Exercise 1 — Electronics Store Inventory System

IElectronicDevice interface

- `void inputDevice();` and `void displayDevice();`

ElectronicDevice class (implements IElectronicDevice)

- Fields: `name` (String), `brand` (String), `releaseYear` (int), `price` (double).
- A no-argument constructor, a constructor with all four fields, and getters/setters for every field.

inputDevice()

- Read every field with `BufferedReader`. Since `readLine()` always returns a `String`, parse `releaseYear` with `Integer.parseInt()` and `price` with `Double.parseDouble()` — both can throw `NumberFormatException` on non-numeric input, which is a different failure from the business-rule validation below and needs its own catch.
- Validate: `name` and `brand` at least 2 characters; `releaseYear` between 2000 and the current year inclusive (get the current year from `java.time.Year.now().getValue()`, not a hard-coded number); `price` > 0.

displayDevice()

- Print all four fields in a readable, labeled format.
- Also call `isPremium()` and print exactly one of: `Device '[Name]' is a premium product.` or `Device '[Name]' is standard.`

isPremium()

- Returns `true` if `price > 1000.0`.

Exception handling in inputDevice()

- Create `InvalidDeviceDataException` extending `Exception`. This is the definitive validation mechanism for inputDevice() — it replaces plain if-checks with actual thrown exceptions, it doesn't sit alongside them as a separate second layer.
- Throw it (with a specific message) whenever a field fails its business-rule check (too-short name/brand, out-of-range year, non-positive price).
- Wrap each field's input in its own try/catch/retry loop — so an invalid price doesn't force the user to re-enter an already-valid name — catching both `InvalidDeviceDataException` (bad value) and `NumberFormatException` (not a number at all) and printing a specific message for each before re-prompting that same field.

InventoryManager class

- Holds an `ArrayList<ElectronicDevice>`.
- `addDevice()`: creates a new `ElectronicDevice`, calls its `inputDevice()`, and adds it to the list.
- `showAllDevices()`: calls `displayDevice()` on every device in the list.
- `findMostExpensive()`: finds and displays the device with the highest price. If the list is empty, show a message instead of an error; if more than one device ties for the highest price, showing any one of them is fine.

Main class and menu:

```
=== ELECTRONICS STORE MENU ===
1. Add New Device
2. Show All Devices
3. Find Most Expensive Device
4. Exit
Your choice:
```

Menu behaviour

- Loop showing the menu; 1–3 call the matching InventoryManager method, 4 exits.

Exercise 2 — Course Management System

Abstract class Course

- Fields: `courseId`, `courseName`, `instructor`, `startDate` (all String), `maxStudents` (int).
- A no-argument constructor and a parameterized constructor; getters/setters for all fields.
- Abstract methods: `public abstract void input(BufferedReader reader) throws IOException;`, `public abstract void display();`, `public abstract String getCourseType();`

CostCalculable interface

- `double getCostMetric();`

OnlineCourse (extends Course, implements CostCalculable)

- Adds `platform` (String), `courseFee` (double).
- No-argument constructor; a full constructor that takes both inherited and new fields and calls `super(...)` for the inherited ones.
- Getters/setters for the two new fields. `input()` prompts for everything (inherited + new) via `BufferedReader`; `display()` prints everything; `getCourseType()` returns `"ONLINE"`; `getCostMetric()` returns `courseFee`.

OfflineCourse (extends Course, implements CostCalculable)

- Adds `roomName` (String), `hasLab` (boolean).
- Same constructor pattern as OnlineCourse, via `super(...)`.
- Getters/setters for the two new fields. `input()` and `display()` follow the same pattern; `getCourseType()` returns `"OFFLINE"`.

OfflineCourse.calculateEstimatedCost()

- Cost per student: `200,000` VND if `hasLab` is true, `120,000` VND if false. Estimated cost = `maxStudents × cost-per-student`.
- After `display()` runs, also print the estimated cost.
- `getCostMetric()` (from CostCalculable) returns this calculated value, not a separately stored field — call calculateEstimatedCost() from inside it.

Data storage and menu

- Use one `ArrayList<Course>` for every course, online or offline — no separate lists per type.
- A CourseManagementTest class with a main method, using BufferedReader throughout:

```
===== COURSE MANAGEMENT SYSTEM =====
1. Add an Online Course
2. Add an Offline Course
3. Display all Courses
4. Display Online Courses (sorted by cost in ascending order)
5. Display Offline Courses (sorted by cost in ascending order)
6. Exit
Your choice:
```

Menu behaviour

- 1/2 — create an `OnlineCourse`/`OfflineCourse`, call `input()`, add it to the shared list.
- 3 — call `display()` on every course in the list, regardless of type.
- 4/5 — filter the shared list down to just one type (either an `instanceof OnlineCourse` check, or comparing `getCourseType()` against `"ONLINE"`/`"OFFLINE"` — both work, since the type-tag method exists for exactly this), sort the filtered results by `getCostMetric()` ascending, and display them.
- 6 — exit.

Exercise 3 — Music Event Management System

Abstract class MusicEvent

- Fields: `eventID` (String), `eventName` (String), `date` (String), `numberOfAttendees` (int).
- A default constructor and a parameterized constructor; getters/setters for all fields; abstract methods `public abstract void input();` and `public abstract void display();`

ConcertEvent (extends MusicEvent)

- Adds `artistName` (String), `genre` (String).
- A no-argument constructor and a 6-argument constructor (4 inherited + these 2), using `super(...)` for the inherited fields.
- Getters/setters for the two new fields. `input()` prompts for every field; `display()` prints every field.

FestivalEvent (extends MusicEvent)

- Adds `numberOfStages` (int), `durationDays` (int).
- Same constructor pattern as ConcertEvent, via `super(...)`.
- Getters/setters for the two new fields. `input()` / `display()` follow the same pattern as ConcertEvent.

calculateEstimatedAttendance()

- New method on `FestivalEvent`: `numberOfStages × durationDays × 1000`.
- After display() runs (as part of menu option 4 below), also print the estimated attendance. Expected result for 3 stages over 2 days:

```
Event ID: EV001
Event Name: Summer Fest
Date: 2023-08-15
Number of Attendees: 5000
Number of Stages: 3
Duration Days: 2
Estimated Attendance: 6000 attendees
```

MusicEventTest class (with main method) and menu:

```
Please select:
1. Input information for n Concert Events.
2. Input information for n Festival Events.
3. Display information of n Concert Events (Sorted by number of attendees descending).
4. Display information of n Festival Events (Sorted by duration days ascending).
5. Exit.
Your choice:
```

Menu behaviour

- 1 — ask how many concert events to enter, then collect that many into a ConcertEvent array.

- 2 — ask how many festival events to enter, then collect that many into a FestivalEvent array.
- 3 — display the ConcertEvent array sorted by number of attendees, descending.
- 4 — display the FestivalEvent array sorted by duration days, ascending (each entry followed by its estimated attendance, per the example above).
- 5 — exit.

Input handling

- Re-prompt rather than crash on invalid keyboard input (e.g. text where a number is expected) — the original only asks for this in general terms without listing specific rules, so beyond "don't crash on bad input," no particular validation range is required here.

Exercise 4 — Employee Management System

IEmployee interface

- `void input();` and `void display();`

Employee class (implements IEmployee)

- Fields: `id` (int, must be unique), `name` (String), `department` (String), `salary` (double, must be > 0).
- Uniqueness for id can't be checked inside Employee itself — a single Employee object has no way to know about any others. Enforce it in EmployeeManagement, at the point a new employee is added to the collection: reject (and re-prompt for) an id that matches one already stored.
- A no-argument constructor and a constructor with all four fields; getters/setters for every field.
- `input()`: read fields with `BufferedReader`; validate name ≥ 5 characters and salary > 0, re-prompting on failure.
- `display()`: print all four fields in a readable format.

calculateTax()

- salary ≤ 50,000 → 10% of salary; 50,000 < salary ≤ 100,000 → 20%; salary > 100,000 → 30%.
- This is a flat rate applied to the whole salary based on which bracket it falls into, **not** a marginal calculation (where different slices of the salary are taxed at different rates, the way real income tax usually works). The given example confirms this: a $120,000 salary produces exactly `$36,000` tax, which is `120,000 × 30%` flat — a marginal calculation would instead give `50,000×10% + 50,000×20% + 20,000×30% = $21,000`, which doesn't match. This is worth getting right deliberately, since "tax bracket" problems default to the marginal interpretation more often than not.
- Print exactly: `Employee [Name] has an annual tax of $[TaxAmount].`

EmployeeManagement class

- `addEmployee()`: collects exactly 3 employees per call, as specified. Keep the underlying array/list as a field that persists and grows by 3 on each call, rather than being replaced — otherwise calling "Add New Employee" a second time would erase the employees entered the first time.
- `showEmployees()`: displays every stored employee.
- `sortEmployeesBySalary()`: sorts the stored employees by salary, ascending.

Main class and menu:

```
=== MENU ===
1. Add New Employee
2. Show All Employees
3. Sort Employees by Salary
```

```
4. Exit
Your choice:
```

Menu behaviour

- 1–3 call the matching EmployeeManagement method; 4 exits.
